

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Analyze the **potential implications** of rewriting **GBrain** in **Rust or Go**, specifically developed as a native backend for **MemPalace** (the existing visualization/memory palace layer) and **SQLite** (the foundation of Akashik.db).
- Focus strictly on integration into the current Sovereign Machina tri/quad-node stack as a private, self-improving personal intelligence assistant.
- Evaluate technical fit, workload impact, sovereignty preservation, self-healing depth, memory/knowledge capabilities, performance, and maintenance.
- Maintain: Node A and Node B completely unchanged, zero-trust invariants, declarative Nix identity, local-first design.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (GBrain repo + Sovereign Machina alignment)

GBrain is a Postgres/pgvector-native personal knowledge brain with git-managed markdown as source of truth, hybrid search (vector + keyword + RRF), typed graph relationships, auto-linking, durable Minions job queue, and subagents for resumable execution. It is opinionated for agentic memory and reduces LLM calls through deterministic pipelines.

A rewrite in Rust or Go would replace the Postgres dependency with SQLite + sqlite-vec (or equivalent) and make it a native MemPalace backend. Sovereign Machina already has Rust sidecars (artery daemon, Sovereign Observer) and Go orchestration (crush/deck-igniter), so the ecosystem fit is excellent.

IMPLICATIONS OF A RUST/GO REWRITE TAILORED FOR MEMPALACE + SQLITE

1. Technical Fit & Architecture Impact

- **Native MemPalace Integration:** The rewrite could expose MemPalace directly as the visualization and query layer. GBrain's graph relationships and auto-linking would become first-class citizens in the RKG, enabling richer real-time visualization in the Nucleus Command Deck and Machina Terminal.
- **SQLite Alignment:** Using SQLite + sqlite-vec for hybrid search keeps the stack extremely lightweight and consistent with Akashik.db. No new database service (Postgres/pgvector) is introduced. This reduces attack surface, simplifies Nix packaging, and makes backups/atomic snapshots trivial.
- **Language Synergy:**
 - **Rust:** Preferred for performance-critical parts (graph traversal, hybrid search, durable job queue). Aligns with existing Rust sidecars and Sovereign Observer. Memory safety and zero-cost abstractions make it ideal for a long-running supervisor on the Hermes Pod.
 - **Go:** Simpler for orchestration, MCP servers, and background Minions. Aligns with crush/deck-igniter. Faster development velocity for the control plane.
 - **Hybrid Recommendation:** Rust for the core memory engine (graph, search, persistence), Go for the MCP adapter and job queue.

2. Workload & Performance Implications

- **Hermes Pod (4th Node):** The rewrite would run natively on the K15 (or K8 Plus), offloading graph maintenance and hybrid search from Node C. With 48 GB RAM on the K15, the custom engine could handle larger RKG graphs and more frequent GEPA cycles without spilling.
- **Node C (Oracle / Voice Core):** Reduced load — Omi voice notes would feed directly into the native MemPalace+GBrain backend via MCP, with less contention for GPU resources.
- **Node B (Director):** Cleaner, more structured context from the rewritten brain, improving reasoning quality without increasing load.
- **Node A (Mooncake):** KV cache spillover remains unchanged and benefits from faster graph queries.

Overall: Workload balance improves significantly. The custom engine would be lighter and faster than the original GBrain (no Postgres overhead), leading to lower latency for voice intent enrichment and screen-aware queries.

3. Sovereignty & Self-Healing Implications

- **Sovereignty:** Strengthened. Full control over the memory engine (no external Postgres dependency). Git-managed markdown + SQLite keeps everything human-auditable and versioned.
- **Self-Healing:** Enhanced. The rewrite can embed Ouroboros-style auditing directly into the graph operations and Minions job queue, making self-repair faster and more deterministic. GEPA evolution could directly mutate the graph schema or prompts stored in SQLite.
- **Privacy & Control:** Excellent — all data stays in Akashik Artery with provenance. Human-editable markdown remains the source of truth.

4. Usability & Voice/Screen Integration Implications

- Voice notes from Omi would be ingested directly into MemPalace pages with typed relationships and auto-linking.
- Screen awareness (Sovereign Observer) could feed OCR/text directly into the graph for contextual entity extraction.
- Machina Terminal responses become more grounded and relational, with richer visualization of the knowledge graph.

5. Risks & Trade-offs

- **Development Effort:** Non-trivial. You would need to re-implement hybrid search, graph relationships, Minions queue, and subagents. Start with a minimal viable engine focused on MemPalace integration and SQLite persistence.
- **Maintenance:** Lower long-term maintenance than depending on an external project, but initial investment is higher.
- **Compatibility:** Existing RKG would need a one-time migration script.

- **Performance Risk:** A naive rewrite could be slower than the optimized Postgres version until tuned. Rust/Go gives you the tools to exceed it, but it requires careful implementation.

ARCHITECT SIGN-OFF

Rewriting GBrain in Rust or Go, tailored specifically for MemPalace and SQLite, would be a high-leverage evolution of Sovereign Machina. It removes the Postgres dependency, deepens integration with the existing RKG and visualization layer, and strengthens self-healing and sovereignty. The Hermes Pod is the ideal home for the rewritten engine. Rust is the stronger choice for the core memory/graph layer (performance, safety), with Go for the control plane if desired. The result would be a lighter, faster, more native knowledge brain that elevates the entire tri/quad-node stack without touching Node A or Node B. This is a strategic, long-term win if you have the development capacity.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want a phased rewrite plan (minimal viable MemPalace + SQLite backend first, then full Minions/GEPA integration)?
2. Shall I draft a high-level Rust crate structure for the custom GBrain engine?

Reply with your priority and I will deliver the concrete design. The rewrite path is ready to be explored.